P1286R2
Contra CWG DR1778

Published Proposal, 2019-02-18

This version:
: http://wg21.link/p1286r2

Author:
: Richard Smith (Google)

Audience:
: CWG, LWG

Project:
: ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper presents a problem with the current resolution of DR1778 and proposes an alternative resolution.

Table of Contents

1 Revision history

2 Background

2.1 CWG 1778: *exception-specification* in explicitly-defaulted functions

2.2 Potential fixes

2.3 Language change

3 Problem

3.1 Existing approach is bad for compilers

3.2 Existing approach is unnecessary for `std::atomic<T>`

3.3 Existing approach prevents a useful feature

4 Approach

5 Wording

References

Informative References

§ 1. Revision history

Since [P1286R1]:

- Addressed CWG wording review comments.
- Addressed LWG wording review comments by removing library wording; the fixes will be left for [LWG2334].

Since [P1286R0]:

- Remove discussion of options; EWG has selected their desired alternative.
- Approved unanimously by EWG.

§ 2. Background

See lists.isocpp.org/core/2018/01/3741.php for more information.

§ 2.1. CWG 1778: *exception-specification* in explicitly-defaulted functions

Prior to [CWG1778], we required that:

> An explicitly-defaulted function [...] may have an explicit *exception-specification* only if it is compatible with the *exception-specification* on the implicit declaration.

It was observed in [LWG2165] that this creates problems for `std::atomic<T>`, which declares its default constructor thusly:

```
template<typename T> struct atomic {
  atomic() noexcept = default;
```

... which (it was believed) resulted in `atomic<T>` being ill-formed if `T` has a potentially-throwing default constructor.

§ 2.2. Potential fixes

[LWG2165] lists the following as potential fixes:

1. Add nothrow default constructible to requirements for template argument of the generic `atomic<T>`
2. Remove `atomic<T>::atomic()` from the overload set if `T` is not nothrow default constructible.
3. Remove `noexcept` from `atomic<T>::atomic()`, allowing it to be deduced (but the default constructor is intended to be always `noexcept`)
4. Do not default `atomic<T>::atomic()` on its first declaration (but makes the default constructor user-provided and so prevents `atomic<T>` being trivial)
5. A core change to allow the mismatched exception specification if the default constructor isn't used (see c++std-core-21990)

§ 2.3. Language change

CWG chose to resolve the issue by changing the rule to:

> If a function that is explicitly defaulted has an explicit *exception-specification* that is not compatible with the *exception-specification* on the implicit declaration, then
>
> - if the function is explicitly defaulted on its first declaration, it is defined as deleted;
> - otherwise, the program is ill-formed.

That is: implicitly delete the default constructor if the specified exception specification doesn't match the implicit one.

§ 3. Problem

§ 3.1. Existing approach is bad for compilers

Exception specifications are a complete-class context: they are a place where all members of the class *and its enclosing classes* can be used, just like member function bodies, default arguments, and default member initializers. This means we cannot in general determine the implicit exception specification of a member function until we reach the end of the outermost lexically-enclosing class. However, we need to know which special member functions a class has, and whether or not they are deleted, immediately after the class becomes complete, which (for a nested class) may be earlier.

Example:

```
struct X { X(); };
struct A {
  struct B {
    B() noexcept(A::value) = default;
    X x;
  };
  decltype(B()) b;
  static constexpr bool value = true;
};
A::B b;
```

Here, we do not parse the exception specification for `B::B()` until after we have finished parsing class `A`. But the class `B` becomes complete at its close brace, and at that point we must know the critical facts regarding its definition, including which of its special members are deleted.

Note that we cannot possibly tell whether the call to `B()` within the `decltype` is valid, because we don’t know whether `A::B::B()` is deleted yet.

§ 3.2. Existing approach is unnecessary for `std::atomic<T>`

The existing core rule arose because of concern over a case such as

```
struct Foo {
  Foo() : n(0) {} // happens to not be noexcept
  int n;
};
std::atomic<Foo> f;
```

... being ill-formed. But it is not: the intent of the `atomic<T>` default constructor is to leave the atomic storage uninitialized, so the `Foo::Foo()` constructor is not invoked, so the implicit exception specification of `atomic<Foo>::atomic()` is always inferred as `noexcept`. The explicit exception specification therefore has no effect.

If the default constructor of `atomic<T>` did default-initialize a `T`, we would need changes here. This problem is to be resolved as part of [LWG2334].

§ 3.3. Existing approach prevents a useful feature

Consider the following pattern, which we found several instances of in our codebase when we tightened up the compiler to reject a mismatched exception specification on a defaulted function:

```
struct X {
  std::map<...> m;
  // ... other members
public:
  // I want a defaulted move constructor, and vector<X> needs to be
  // efficient, so please call std::terminate if moving the map throws
  // rather than slowing my code down with unnecessary copies
  X(X &&) noexcept = default;
};
```

Users wanting this feature are forced to write out their own special members, which is an error-prone operation that `= default` was supposed to alleviate.

§ 4. Approach

If the user explicitly specifies an exception specification on a defaulted function, that’s the exception specification. Don’t delete the function, don’t reject the program, just accept it.

§ 5. Wording

Change in [dcl.fct.def.default]/2:

> The type `T1` of an explicitly defaulted function `F` is allowed to differ from the type `T2` it would have had if it were implicitly declared, as follows:
>
> - `T1` and `T2` may have differing *ref-qualifiers*; and
> - `T1` and `T2` may have differing exception specifications; and
> - if `T2` has a parameter of type `const C&`, the corresponding parameter of `T1` may be of type `C&`.
>
> [...]

Change in [dcl.fct.def.default]/4:

> `~S() noexcept(false) = default;` // ~~deleted: exception specification does not match~~ OK, despite mismatched exception specification

Add the following to the example in [dcl.fct.def.default]/4:

```
struct T { T(); T(T &&) noexcept(false); };
struct U { T t; U(); U(U &&) noexcept = default; };
U u1;
U u2 = static_cast<U&&>(u1);       // OK, calls std::terminate if T::T(T&
```

Do not make any changes to [atomics.types].

§ References

§ Informative References

[CWG1778]

USA. exception-specification in explicitly-defaulted functions. 25 September 2013. C++14. URL: https://wg21.link/cwg1778

[LWG2165]
Jonathan Wakely. std::atomic<X> requires X to be nothrow default constructible. Resolved. URL: https://wg21.link/lwg2165

[LWG2334]
Daniel Krügler. atomic's default constructor requires "uninitialized" state even for types with non-trivial default-constructor. SG1. URL: https://wg21.link/lwg2334

[P1286R0]
Richard Smith. Contra CWG DR1778. 5 October 2018. URL: https://wg21.link/p1286r0

[P1286R1]
Richard Smith. Contra CWG DR1778. 18 January 2019. URL: https://wg21.link/p1286r1